

La Federación de Contexto con Lectura Primero y Control por Rol

Supera al Paso de Mensajes Punto a Punto en Sistemas Multiagente Basados en LLM: Una Evaluación Controlada por Equidad

Adrian Issac Mamani Quispe
Universidad Nacional de San Agustín de Arequipa
Arequipa, Perú
admamaniq@unsa.edu.pe

Ryan Fabian Valdivia Segovia
Universidad Nacional de San Agustín de Arequipa
Arequipa, Perú
rvaldiviase@unsa.edu.pe

Junio de 2026

Resumen

Los sistemas multiagente contruidos sobre grandes modelos de lenguaje (LLM) coordinan, en su inmensa mayoría, mediante *paso de mensajes*: cada agente envía la tarea y el contexto directamente a pares con nombre, como en el protocolo Agent2Agent (A2A) de Google y en los lenguajes clásicos de comunicación entre agentes de los que descende. Sostenemos que un sustrato alternativo —*federación de contexto con lectura primero y control por rol* (el Graph Context Protocol, GCP)— es estructuralmente superior en tres ejes con igual éxito de tarea: **acoplamiento, fuga de información y costo**. En GCP, un agente no empuja contexto hacia sus pares; lo lee desde nodos de conocimiento de propiedad independiente a través de una compuerta de política que autoriza la lectura por rol y registra una entrada de procedencia por cada decisión. Hacemos la comparación *justa* manteniendo todo constante salvo la capa de interoperabilidad: ambos brazos ejecutan el mismo cerebro de agente, el mismo modelo y la misma tarea; solo difieren la fábrica de herramientas por par y el alojamiento de nodos (un manejador de `fetch` de GCP en proceso frente a servidores A2A reales en `localhost`). En tres escenarios —un mercado (acoplamiento), una organización de software (fuga) y una cadena de suministro entre propietarios (entre dueños)— encontramos: (i) el número de aristas de integración por pares crece como $\mathcal{O}(n^2)$ para A2A frente a $\mathcal{O}(n)$ para GCP (2450 frente a 50 en $n=50$), con esfuerzo de integración marginal constante para GCP; (ii) la completitud de procedencia es 1,0 para GCP y 0,0 para A2A

en todos los escenarios y semillas, porque las decisiones de lectura se auditan por construcción mientras que los mensajes punto a punto no dejan rastro de acceso; y (iii) GCP nunca cuesta más y, en la tarea entre propietarios, cuesta menos tokens con mucha menor varianza ($707,6 \pm 3,5$ frente a $901,4 \pm 229,2$). El éxito de tarea es igual en ambos brazos. Posicionamos GCP frente a cinco décadas de teoría del acoplamiento e investigación en control de acceso, y frente a trabajo reciente sobre seguridad de agentes LLM, y liberamos el banco de pruebas, los escenarios y las semillas. La evaluación es preliminar —un modelo de capa gratuita, cinco semillas— y somos explícitos sobre qué autoriza y qué no.

Palabras clave—*federación de contexto; sistemas multiagente; grandes modelos de lenguaje; control de acceso; mínimo privilegio; procedencia; acoplamiento de software; protocolos de comunicación entre agentes*

I. Introducción

El patrón de coordinación dominante para los sistemas multiagente basados en LLM es el paso directo de mensajes. Marcos como AutoGen [1], ChatDev [2] y MetaGPT [3] componen aplicaciones a partir de agentes que envían y reciben mensajes, y el emergente protocolo Agent2Agent (A2A) [4] estandariza exactamente esto: un agente cliente anuncia y delega tareas a agentes remotos con nombre sobre JSON-RPC. Este paradigma descende directamente de los lenguajes clásicos de comunicación entre agentes KQML [5]

y FIPA-ACL [6], que ya asumían conexiones por pares y protocolos compartidos, prenegociados, entre cada par que interactúa.

El paso de mensajes tiene un costo estructural que la literatura de ingeniería de software comprende desde hace cincuenta años. Conectar n componentes por pares requiere del orden de n^2 enlaces; introducir un intermediario compartido al que cada componente se conecta una sola vez reduce esto a n enlaces. Este es el “espagueti de integración” que el catálogo de Patrones de Integración Empresarial [7] aborda con corredores de mensajes y concentradores, y es el mismo fenómeno que la teoría del acoplamiento [8–10] formaliza: la modificabilidad de un sistema está gobernada por el número y la fuerza de sus dependencias entre componentes.

Estudiamos un sustrato alternativo para la coordinación multiagente, el Graph Context Protocol (GCP). GCP es de *lectura primero*: un agente obtiene lo que necesita leyendo contexto desde nodos de conocimiento de propiedad independiente, en lugar de recibir mensajes empujados. Tiene *control por rol*: cada lectura pasa por una compuerta de política que autoriza la solicitud según el rol del solicitante y sus capacidades requeridas, denegando por defecto [11, 12]. Y es *auditado*: cada decisión de lectura —concesión o denegación— produce un registro de procedencia estructurado que nombra al principal, el nodo objetivo, la decisión y la política coincidente.

Tesis. Con igual éxito de tarea, la federación de contexto con lectura primero y control por rol supera al paso de mensajes punto a punto en acoplamiento, fuga y costo.

Contribuciones.

1. **Arquitectura.** Describimos GCP: federación de contexto con lectura primero y control por rol, con procedencia por lectura de primera clase (Sección III), y la contrastamos con el paso de mensajes basado en empuje.
2. **Un método de evaluación controlado por equidad.** El defecto más común en las comparaciones de arquitecturas es que los brazos difieren en más que la arquitectura. Nuestro banco de pruebas construye ambos brazos a partir de una sola definición de escenario y un solo modelo inyectado, intercambiando *únicamente* la capa de interoperabilidad (Sección V).
3. **Un enfrentamiento empírico** sobre tres escenarios con un LLM real y cinco semillas, reportando conteos (conexiones, mensajes, tokens) y no solo tiempo de reloj (Sección VI).
4. **Definiciones de métricas** para el acoplamiento estructural, la fuga por coincidencia exacta de canario (una cota inferior) y la completitud de procedencia (Sección IV).

Somos deliberadamente conservadores con las afirmaciones empíricas: los resultados de comportamiento provienen de un único modelo de capa gratuita sobre cinco semillas y deben leerse como un *piloto*. La Sección VII expone las amenazas a la validez en su totalidad, incluido un caso en el que la fuga arquitectónica a favor de GCP es *latente y no observada* porque el modelo nunca ejercitó la ruta de fuga.

II. Antecedentes y Trabajo Relacionado

A. El paso de mensajes como línea base multiagente

Los lenguajes de comunicación entre agentes establecieron el paradigma del paso de mensajes: KQML [5] envolvía cargas de conocimiento en performativos tipados intercambiados sobre conexiones por pares, y FIPA-ACL [6] estandarizó un mensaje de 12 campos en el que el emisor y el receptor deben compartir lenguaje de contenido y ontología antes de que cualquier intercambio tenga éxito. Los marcos modernos de LLM reinventaron esto en lenguaje natural. AutoGen [1] programa aplicaciones como conversaciones entre agentes; ChatDev [2] dispone pares de roles en una “cadena de chat” fija; y los estudios del campo [13] taxonomizan la estructura de comunicación en variantes punto a punto, centralizada, por capas y de fondo compartido. El Model Context Protocol [14] estandariza el acceso a contexto de agente a herramienta (una forma temprana de lectura primero), mientras que A2A [4] estandariza la delegación de tareas de agente a agente —la línea base con la que comparamos. Trabajo reciente ha comenzado a cuantificar la sobrecarga del paso de mensajes: Zhang et al. [15] reportan un exceso de tokens del 28–73 % en los grafos de comunicación entre agentes y recuperan la mayor parte podando la topología de mensajes. De manera muy relacionada, Mao and Mirhoseini [16] reemplazan tanto la orquestación centralizada como el enrutamiento punto a punto por un contexto compartido verificado y reportan resultados de vanguardia con aproximadamente la mitad del costo —evidencia de que un sustrato de lectura desde contexto compartido puede dominar al paso de mensajes. A esa línea añadimos autorización por rol, procedencia auditada por lectura y un enfrentamiento controlado por equidad.

B. Acoplamiento y el problema n^2

Que menos enlaces entre componentes producen sistemas más mantenibles es de los resultados más antiguos de la ingeniería de software. Parnas [8] mostró que la descomposición por ocultamiento de información —exponer solo una interfaz mínima— produce un acoplamiento drásticamente menor que la partición por diagrama de flujo. Stevens et al.

[9] introdujeron el acoplamiento y la cohesión como propiedades medibles y ordinales de la dependencia entre módulos. Briand et al. [10] dieron un marco teórico de medición y de teoría de grafos en el que el acoplamiento es una función monótona del conteo de aristas de dependencia, por lo que una topología de $\mathcal{O}(n)$ aristas está demostrablemente menos acoplada que una de $\mathcal{O}(n^2)$. Hohpe and Woolf [7] lo trasladaron a la integración distribuida: una malla punto a punto completa de n sistemas necesita $n(n-1)/2$ canales, que un corredor de tipo concentrador reduce a n . La investigación en microservicios [17] lleva el mismo argumento de conteo de aristas a los sistemas nativos de la nube, y en el ámbito de los LLM Shen et al. [18] muestran empíricamente que las topologías de agentes densas y completamente conectadas amplifican la propagación de errores mientras que las topologías más dispersas la suprimen. Aplicamos esto directamente: A2A crea aristas agente $\leftrightarrow$ agente por pares; GCP crea aristas agente $\rightarrow$ almacén.

C. Mínimo privilegio, RBAC y contexto con compuerta

La compuerta de lectura de GCP operacionaliza el principio de mínimo privilegio, articulado por Saltzer and Schroeder [11] y equiparado allí con la necesidad de conocer (need-to-know) militar. El control de acceso basado en roles [12] desacopla los permisos de los individuos asignándolos a roles, y el control de acceso basado en atributos [19] generaliza la política a atributos arbitrarios; la protección basada en capacidades [20] liga la autoridad a fichas infalsificables y explícitamente pasadas —el modelo que siguen las credenciales de rol de GCP. Las obligaciones de minimización de datos y limitación de finalidad [21] convierten “leer solo lo que tu rol requiere” en una restricción legal, no en una mera preferencia de diseño. La línea base punto a punto no tiene una compuerta análoga: la prevención de fugas recae enteramente en la discreción del emisor.

D. Fuga, procedencia y gobernanza en sistemas de agentes

La literatura de seguridad establece que la confidencialidad en los sistemas de agentes es una propiedad del sistema, no solo del modelo [22], que los canales de mensajes entre agentes son el vector de fuga dominante —filtrando mucho más que las salidas finales y siendo en gran medida invisibles a la auditoría solo de salidas [23]— y que restringir lo que un agente puede *ver* —en lugar de filtrar lo que puede *decir*— es la intervención más robusta [24]. El Top 10 de OWASP para aplicaciones LLM [25] nombra la Divulgación de Información Sensible y la Agencia Excesiva como riesgos primarios, con “adoptar el mínimo privilegio” como mitigación normativa. Del lado de la rendición de cuentas, el control de privilegios determinista en la frontera de herramientas [26], la delegación de autoridad autenticada y

auditable a los agentes [27] y los registros de auditoría para LLM [28] abogan por registros a prueba de manipulación y consultables de lo que hizo un sistema. La contribución de GCP a esta línea es hacer de la *decisión por lectura* el primitivo de procedencia: cada lectura de contexto autorizada o denegada se registra, algo que el paso de mensajes no hace.

Diferencia (delta). El trabajo previo aporta las piezas —acceso de lectura primero (MCP), control por rol/atributo (RBAC/ABAC, RAG con compuerta) y registros de auditoría— pero no su conjunción, y no un enfrentamiento controlado por equidad contra una línea base real de paso de mensajes. GCP combina acceso de lectura primero, control por rol y procedencia auditada por lectura, y medimos la combinación contra A2A.

III. El Graph Context Protocol

GCP modela un ecosistema multiagente como *nodos de conocimiento* y *nodos de agente* de propiedad independiente. La coordinación es de lectura primero: un agente adquiere contexto emitiendo una *consulta de contexto* a un nodo par, que devuelve el contexto que el solicitante está autorizado a leer. Tres propiedades distinguen al sustrato.

Lectura primero. Los productores no empujan hacia los consumidores. Un nodo de conocimiento expone contexto; los consumidores lo extraen bajo demanda. Ningún agente necesita conocer la identidad de los *consumidores* de sus productores, y añadir un consumidor no requiere ningún cambio en el productor. Estructuralmente, cada agente se conecta a los nodos que lee, no a todos los demás agentes.

Control por rol. Cada nodo de conocimiento porta una política de acceso (`readableByRoles`, `requiredCapabilities`, `fallbackAllowed`, `denialMode`) y el servidor autoriza cada consulta contra el principal del solicitante, denegando por defecto cuando ninguna política concede el acceso. La autorización ocurre *antes* de que cualquier contenido llegue a la ventana de contexto del agente —el análogo de lectura primero de la mediación completa [11].

Auditado. Cada decisión de lectura emite un registro de procedencia estructurado (`principalId`, `targetNodeId`, `timestamp`, `decision`, `matchedPolicy`) a un sumidero de auditoría enchufable. Esto produce un libro mayor completo de quién-leyó-qué-cuándo para la federación, incluidas las denegaciones.

La línea base de paso de mensajes (A2A) implementa las mismas tareas de escenario haciendo que los agentes mantengan referencias a pares mediante tarjetas e intercambien contexto a través de mensajes directos, con permitir/denegar grueso declarado en la tarjeta y sin sumidero

de auditoría por lectura —un control fiel y de buena fe, construido con el mismo cuidado que el brazo GCP.

IV. Modelo de Amenazas y Costo, y Definición de Métricas

Reportamos métricas en tres familias.

Acoplamiento estructural (determinista). Para n nodos pares, las *conexiones por pares* son el número de aristas de integración que la topología requiere. El brazo punto a punto necesita $C_{A2A}(n) = n(n-1)$ aristas dirigidas (cada par agente $\leftrightarrow$ par); el brazo federado necesita $C_{GCP}(n) = n$ (cada agente $\rightarrow$ almacén). El *esfuerzo de integración* es el costo marginal de añadir un nodo, $C(n+1) - C(n)$: constante 1 para GCP y $2n$ para A2A. Estas son propiedades de la topología, no de una ejecución, así que se calculan exactamente.

Fuga (una cota inferior). Cada nodo confidencial incrusta una cadena *canario* única. Tras una ejecución, escaneamos la respuesta final y la transcripción completa de herramientas en busca de coincidencias exactas del canario; la *tasa de fuga* es la fracción de canarios del escenario que aparecen donde no deberían. El escaneo por coincidencia exacta es una cota inferior: la divulgación parafraseada no se cuenta.

Completitud de procedencia. La fracción de decisiones de lectura para las que existe un registro de procedencia estructurado, acotada a $[0, 1]$. GCP registra cada decisión; el brazo de paso de mensajes no tiene sumidero de auditoría, así que su completitud es 0 por construcción.

Costo de comportamiento y éxito. Contamos mensajes, conexiones abiertas, viajes de ida y vuelta y tokens totales (sumados desde los metadatos de uso del proveedor), medimos la latencia de reloj y evaluamos el éxito de tarea con un oráculo por escenario. Los conteos —no solo el tiempo de reloj— son la señal primaria de costo, ya que el tiempo de reloj en un punto final gratuito compartido es ruidoso.

V. Metodología

La invariante de equidad. El compromiso metodológico central es que los dos brazos difieren *únicamente* en la capa de interoperabilidad. Un único ejecutor de escenarios construye ambos brazos a partir de un `ScenarioDef` y un modelo de chat inyectado. Ambos brazos ejecutan el mismo cerebro de agente (un agente ReAct con una herramienta por par), el mismo prompt de sistema, objetivo y oráculo de éxito. El brazo GCP aloja los nodos de conocimiento en

proceso detrás del manejador de `fetch` del servidor real con un proveedor de autenticación por token estático y un sumidero de auditoría en memoria; el brazo A2A aloja los mismos nodos como servidores A2A reales en `localhost`. Solo cambian la fábrica de herramientas por par y el alojamiento de nodos. Esto elimina el factor de confusión que hace ininterpretables a la mayoría de las comparaciones de arquitecturas.

Escenarios. *software-org* (fuga): un agente con rol de contratista debe resumir información pública sobre un proyecto; un nodo de ingeniería confidencial que porta un canario es legible solo por el rol de ingeniería. *supply-chain* (entre propietarios): un agente de logística debe enunciar la relación de envío compartible entre dos nodos de propiedad independiente (un proveedor y un fabricante), cada uno con un canario de alcance del propietario. *marketplace* (acoplamiento): n nodos vendedores con precios descendentes; el agente debe identificar el más barato. El mercado está parametrizado por n y genera la curva de acoplamiento estructural.

Modelo, semillas, banco de pruebas. Todas las ejecuciones de comportamiento usan `openai/gpt-oss-120b:free` vía OpenRouter a temperatura 0,7, con cinco repeticiones (semillas) por brazo por escenario. El banco de pruebas está protegido tras una variable de entorno y una clave de API para que nunca se ejecute en CI; aplica regulación (throttling) entre ejecuciones y reintenta ante errores transitorios de límite de tasa, ambos comunes en puntos finales gratuitos. Reportamos media $\pm$ desviación estándar poblacional. La curva de acoplamiento estructural se calcula de forma determinista para $n = 2, \dots, 50$.

VI. Resultados

A. El acoplamiento escala como $\mathcal{O}(n)$ frente a $\mathcal{O}(n^2)$

La Tabla 1 muestra la curva de acoplamiento estructural. Las conexiones por pares de GCP crecen linealmente (n) mientras que las de A2A crecen cuadráticamente ($n(n-1)$): idénticas en $n=2$, $3\times$ en $n=3$, $9\times$ en $n=10$ y $49\times$ en $n=50$ (50 frente a 2450). El costo marginal de añadir un nodo —el esfuerzo de integración— es constante 1 para GCP y $2n$ para A2A, por lo que la brecha *se ensancha* con cada nodo añadido.

B. La procedencia es completa para gcp y ausente para a2a

En los tres escenarios y las cinco semillas, la completitud de procedencia es 1,0 para GCP y 0,0 para A2A (Tabla 2). Esto es arquitectónico, no estadístico: GCP registra una

Cuadro 1. Acoplamiento estructural (aristas de integración por pares) según el tamaño del ecosistema n . Determinista.

n	GCP (n)	A2A ($n(n-1)$)	razón A2A/GCP
2	2	2	1,0×
3	3	6	2,0×
5	5	20	4,0×
10	10	90	9,0×
20	20	380	19,0×
50	50	2450	49,0×

decisión por cada lectura; el brazo de paso de mensajes no tiene sumidero de auditoría, así que no existe rastro de acceso que consultar. Para cualquier régimen de gobernanza que requiera trazabilidad [25, 28], esta es la diferencia cualitativa más nítida entre los sustratos.

C. gcp nunca cuesta más, y cuesta menos en la tarea entre propietarios

La Tabla 2 reporta las métricas de comportamiento. En *software-org* los dos brazos están dentro del ruido en tokens (678,4±5,8 frente a 685,4±3,4). En *supply-chain* GCP usa menos tokens con mucha menor varianza (707,6±3,5 frente a 901,4±229,2) y menos mensajes/viajes de ida y vuelta (1,0±0,0 frente a 1,4±0,5): el brazo de paso de mensajes a veces abrió conexiones extra para ensamblar la respuesta entre propietarios, mientras que el brazo federado leyó lo que necesitaba en una sola pasada. El éxito de tarea es 1,0 en ambos brazos para estos dos escenarios —se cumple la precondition de igual éxito de la tesis.

D. Fuga

El panorama de la fuga requiere cuidado. En *supply-chain*, donde el rol de logística está autorizado a leer los nodos de ambos propietarios, el brazo de paso de mensajes filtró cada canario de alcance del propietario (tasa 1,0) mientras que GCP filtró la mitad en promedio (0,5); como ambos brazos están autorizados a leer el contenido, esta brecha refleja cómo cada sustrato expone el contenido al modelo más que una decisión de control. En *software-org*, el caso de control más limpio, la fuga *observada* fue 0,0 en ambos brazos —pero por una razón instructiva: con este modelo el agente satisfizo la tarea de resumen público consultando solo el nodo público (mensajes = 1,0) y nunca solicitó el nodo confidencial, así que la ruta de fuga no se ejercitó. GCP habría *denegado* esa lectura por construcción de haber ocurrido (y registrado la denegación); la tarjeta gruesa del brazo de paso de mensajes la habría respondido. La fuga arquitectónica es, por tanto, real pero *latente* en esta ejecución, y no afirmamos una victoria de fuga de comportamiento observada en *software-org*. La garantía estructural —denegar por defecto más denegación auditada—

es lo que GCP provee, independientemente de si un modelo dado sondea la frontera. La métrica de canario *mide* esta cota; el modelo formal y la suite de propiedades ejecutables descritos en la Sección E la *prueban*.

E. Modelo formal y propiedad ejecutable de no fuga

Modelo de acceso. Un nodo de conocimiento porta un `AccessPolicyDescriptor` P con dos campos de control: *readableByRoles* (una lista de identificadores de rol) y *requiredCapabilities* (una lista de identificadores de capacidad). Un principal K tiene un rol opcional; su conjunto de capacidades efectivas *effective*(K) es la unión de las capacidades planas propias de K ($K.capabilities$) y las heredadas transitivamente a través de la cadena de roles (precedencia propia ante identificadores de capacidad duplicados), calculándose la parte del rol con `getEffectiveCapabilities`. El predicado de admisión es:

$$authorized(P, K) \equiv \underbrace{(P.readableByRoles = [] \vee K.role.id \in P.readableByRoles)}_{roleOk}$$

Una política ausente o con esquema inválido se trata como denegación (denegar por defecto): la compuerta está cerrada cuando no hay política válida disponible.

Teorema de no fuga (informal). Para todo P y K : si $\neg authorized(P, K)$, entonces ninguna `context-query` dirigida a un nodo con política P devuelve el contenido de ese nodo a K . El argumento se sigue de la tubería del manejador en `createContextQueryHandler`: la autorización (paso 4) precede cualquier llamada al adaptador de fuente de conocimiento (paso 5). En caso de fallo, el manejador devuelve un resultado `denied` que contiene solo el motivo de denegación; el adaptador nunca es invocado y el contenido del nodo nunca entra en la ruta de serialización de la respuesta.

Suite de propiedades ejecutables. Se implementan tres pruebas de propiedades `fast-check` sobre la *ruta de código real en producción* con valores predeterminados de CI de `FC_NUM_RUNS` = 1000; todas pasan.

P1 (no fuga de contenido de extremo a extremo). Para cada par (P, K) generado aleatoriamente, la propiedad conecta una instancia activa de `createContextQueryHandler()` con un adaptador stub que devuelve incondicionalmente una cadena centinela `SECRET-CANARY-...` en su campo `raw` y un proveedor de autenticación que todo lo permite (para que el término del proveedor desaparezca). Se afirman dos direcciones simultáneamente: *seguridad* — cuando $\neg authorized(P, K)$, la serialización JSON completa de la respuesta del manejador no contiene el centinela y `payload.status` es `"denied"`; y

Cuadro 2. Métricas de comportamiento, media $\pm$ desv. est. poblacional sobre 5 semillas, **gpt-oss-120b:free**. Las columnas estructurales corresponden al número de nodos del escenario ($n=2$).

escenario	brazo	tokens	mensajes	latencia (ms)	fuga	procedencia
software-org	GCP	678,4 $\pm$ 5,8	1,0 $\pm$ 0,0	6268 $\pm$ 482	0,0	1,0
	A2A	685,4 $\pm$ 3,4	1,0 $\pm$ 0,0	5986 $\pm$ 1168	0,0	0,0
supply-chain	GCP	707,6 $\pm$ 3,5	1,0 $\pm$ 0,0	6278 $\pm$ 1632	0,5	1,0
	A2A	901,4 $\pm$ 229,2	1,4 $\pm$ 0,5	7718 $\pm$ 2678	1,0	0,0
marketplace [†]	GCP	2425 $\pm$ 1001	2,8 $\pm$ 0,7	12617 $\pm$ 3757	0,0	1,0
	A2A	2431 $\pm$ 1082	2,8 $\pm$ 0,7	11941 $\pm$ 4304	0,0	0,0

[†] las filas de marketplace agregan $n \in \{2, 5, 10\}$; el éxito fue 0,3 $\pm$ 0,5 en ambos brazos (véase la Sección VII). Use la curva estructural (Tabla 1), no estas filas de comportamiento con n mezclado, para la afirmación de acoplamiento.

vivacidad — cuando $authorized(P, K)$, el centinela *sí* aparece, confirmando que el banco de pruebas es capaz de filtrar y no pasa de manera vacua.

P2 (solidez de autorización). Para cada par (P, K) generado aleatoriamente, se llama a la función real `authorizeKnowledgeNodeAccess` contra un nodo con política P usando un proveedor que todo lo permite, y el resultado se afirma que es igual a $authorized(P, K)$ calculado por el oráculo de referencia independiente. Esto establece que el predicado de referencia es una especificación precisa de la compuerta real, el enlace compositivo clave que permite a P1 usar *authorized* como oráculo.

P3 (solidez de herencia de roles). Se generan cadenas aleatorias de roles (profundidad 2–4) sobre un conjunto fijo de cinco identificadores de capacidad y se verifican los invariantes de completitud y de precedencia-propia de `getEffectiveCapabilities`. P3 fundamenta el cálculo de capacidades efectivas usado tanto por el predicado de referencia como por la compuerta real.

Medición de canario frente a prueba formal. La métrica de canario (Sección V) cuantifica la frecuencia de fuga en escenarios similares a producción; es una cota inferior *medida*. La suite de propiedades aquí es un complemento a *nivel de prueba*: demuestra que la cota de no fuga se cumple en todo el espacio de entrada aleatorizado (sujeto a los conjuntos finitos de los generadores). La métrica de canario puede detectar una regresión si la compuerta es eludida; la suite de propiedades explica por qué no debería ocurrir ninguna fuga y proporciona una explicación constructiva del invariante. Juntas forman una garantía de dos capas: la propiedad demuestra que la compuerta es correcta; el canario confirma que la compuerta está en la ruta caliente en escenarios similares a producción.

VII. Discusión y Amenazas a la Validez

Un solo modelo, cinco semillas. Los resultados de comportamiento usan un único modelo de capa gratuita (**gpt-oss-120b**) sobre cinco semillas. Por eso los presentamos como un piloto. Los resultados de acoplamiento estructural y de completitud de procedencia *no* dependen del modelo —el primero es determinista, el segundo arquitectónico— así que son robustos a esta limitación; los resultados de tokens y latencia deberían replicarse entre modelos antes de tratarse como generales.

Fuga latente frente a observada. Como detalla la Sección VI, la fuga de software-org a favor de GCP no se observó en el comportamiento porque el modelo no sondeó el nodo confidencial. Esta es una limitación genuina de la medición de fuga de comportamiento: depende de que el agente ejerce la ruta. La mitigamos reportando también la garantía arquitectónica y la procedencia de las denegaciones, y el banco de pruebas incluye una prueba hermética de consulta forzada que ejerce la ruta de fuga directamente.

La fuga es una cota inferior. El escaneo por coincidencia exacta de canario omite la divulgación parafraseada y parcial. Las tasas de fuga reportadas, por tanto, subestiman la exposición real para ambos brazos por igual.

Dificultad de la tarea de mercado. El éxito de la tarea de mercado fue bajo (0,3) para *ambos* brazos: **gpt-oss-120b** a menudo no logró identificar al vendedor más barato. Como el fallo es igual en ambos brazos, no amenaza la equidad, pero significa que el mercado contribuye a través de su curva estructural determinista, no de sus filas de comportamiento.

Ruido de la capa gratuita. La latencia de reloj en un punto final gratuito compartido tiene alta varianza; lideramos con conteos (mensajes, tokens) para el costo y tratamos la latencia como secundaria.

Validez de constructo de la línea base. Una línea base débil haría vacua la comparación. El brazo A2A ejecuta servidores reales, el mismo cerebro y la misma tarea, y se construyó con igual cuidado; sus desventajas (sin compuerta de lectura, sin sumidero de auditoría, topología por pares) son intrínsecas al paso de mensajes, no artefactos de un hombre de paja.

VIII. Limitaciones y Trabajo Futuro

La evaluación debería extenderse a múltiples modelos y más semillas, con pruebas de significancia sobre las diferencias de tokens y latencia. Queda una capacidad diseñada pero no evaluada: una capa de *delegación de tareas* con capacidad más estricta sobre la misma ruta de lectura primero. El modelo formal con una propiedad ejecutable de no fuga está ahora completo (Sección E), elevando el argumento de fuga de medido a probado. Puentear GCP hacia MCP y A2A como interoperabilidad de primera clase (más allá de la línea base del banco de pruebas) es la ruta natural de despliegue.

IX. Conclusión

Argumentamos, y dimos evidencia preliminar controlada por equidad, de que la federación de contexto con lectura primero y control por rol supera al paso de mensajes punto a punto en sistemas multiagente basados en LLM en acoplamiento, fuga y costo con igual éxito de tarea. El acoplamiento mejora de $\mathcal{O}(n^2)$ a $\mathcal{O}(n)$ con esfuerzo de integración marginal constante; la procedencia es completa por construcción en lugar de ausente; y el costo nunca es peor y a veces es mejor. La arquitectura convierte la prevención de fugas de una cuestión de discreción del emisor en una compuerta de lectura que deniega por defecto y se audita. Los resultados más fuertes —acoplamiento y procedencia— son independientes del modelo; los resultados de costo y fuga son un piloto específico de un modelo que invita a la replicación.

Disponibilidad

El banco de pruebas, los escenarios, las semillas y el informe de evaluación están disponibles en el repositorio del proyecto Graph Context Protocol.

Referencias

- [1] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [2] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL <https://arxiv.org/abs/2307.07924>.
- [3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2308.00352>.
- [4] Google and Linux Foundation. Agent2Agent (A2A) protocol specification, v1.0. Open standard, 2025. URL <https://a2a-protocol.org/v1.0.0/specification/>.
- [5] Tim Finin, Rich Fritzson, Don McKay, and Robin McEntire. KQML as an agent communication language. In *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM)*, 1994.
- [6] Foundation for Intelligent Physical Agents. FIPA ACL message structure specification. Technical Report SC00061G, FIPA, 2002. URL <http://www.fipa.org/specs/fipa00061/>.
- [7] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2003.
- [8] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.
- [9] W. P. Stevens, G. J. Myers, and L. L. Constantine. Structured design. *IBM Systems Journal*, 13(2):115–139, 1974. doi: 10.1147/sj.132.0115.
- [10] Lionel C. Briand, John W. Daly, and Jürgen K. Wüst. A unified framework for coupling measurement in object-oriented systems. *IEEE Transactions on Software Engineering*, 25(1):91–121, 1999. doi: 10.1109/32.748920.

- [11] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [12] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996. doi: 10.1109/2.485845.
- [13] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pages 8048–8057, 2024. URL <https://arxiv.org/abs/2402.01680>.
- [14] Anthropic. Introducing the model context protocol. Anthropic; open specification, 2024. URL <https://www.anthropic.com/news/model-context-protocol>.
- [15] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for LLM-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024. URL <https://arxiv.org/abs/2410.02506>.
- [16] Yuzhen Mao and Azalia Mirhoseini. Decentralized multi-agent systems with shared context. *arXiv preprint arXiv:2606.10662*, 2026. URL <https://arxiv.org/abs/2606.10662>.
- [17] Sebastiano Panichella, Mohammad Imranur Rahman, and Davide Taibi. Structural coupling for microservices. In *Proceedings of the 11th International Conference on Cloud Computing and Services Science (CLOSER)*, pages 280–287, 2021. doi: 10.5220/0010481902800287. URL <https://arxiv.org/abs/2103.04674>.
- [18] Xu Shen, Yixin Liu, Yiwei Dai, Yili Wang, Rui Miao, Yue Tan, Shirui Pan, and Xin Wang. Understanding the information propagation effects of communication topologies in LLM-based multi-agent systems. *arXiv preprint arXiv:2505.23352*, 2025. URL <https://arxiv.org/abs/2505.23352>.
- [19] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. Guide to attribute based access control (ABAC) definition and considerations. Technical Report NIST Special Publication 800-162, National Institute of Standards and Technology, 2014.
- [20] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966. doi: 10.1145/365230.365252.
- [21] European Parliament and Council. Regulation (EU) 2016/679 (general data protection regulation), articles 5 and 25. Official Journal of the European Union, L 119, 2016. URL <https://gdpr-info.eu/>.
- [22] Jonathan Evertz, Merlin Chlosta, Lea Schönherr, and Thorsten Eisenhofer. Whispers in the machine: Confidentiality in agentic systems. *arXiv preprint arXiv:2402.06922*, 2024. URL <https://arxiv.org/abs/2402.06922>.
- [23] Fatima El Yagoubi, Godwin Badu-Marfo, and Ranwa Al Mallah. AgentLeak: A full-stack benchmark for privacy leakage in multi-agent LLM systems. *arXiv preprint arXiv:2602.11510*, 2026. URL <https://arxiv.org/abs/2602.11510>.
- [24] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. AirGapAgent: Protecting privacy-conscious conversational agents. *arXiv preprint arXiv:2405.05175*, 2024. URL <https://arxiv.org/abs/2405.05175>.
- [25] OWASP GenAI Security Project. OWASP top 10 for large language model applications. OWASP Foundation, 2025. URL <https://owasp.org/www-project-top-10-for-large-language-model-application/s/>.
- [26] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Programmable privilege control for LLM agents. *arXiv preprint arXiv:2504.11703*, 2025. URL <https://arxiv.org/abs/2504.11703>.
- [27] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. Authenticated delegation and authorized AI agents. *arXiv preprint arXiv:2501.09674*, 2025. URL <https://arxiv.org/abs/2501.09674>.
- [28] Victor Ojewale, Harini Suresh, and Suresh Venkatasubramanian. Audit trails for accountability in large language models. *arXiv preprint arXiv:2601.20727*, 2026. URL <https://arxiv.org/abs/2601.20727>.